

Phase II Source Code Report

CMSC 495 Capstone Project Group 4

<https://github.com/metersploit/CMSC495.git>

Team Members

Welsh, Gavyn

Maxey, Brandon

Smith, Natquane

Rogers, Eric

1. Introduction

For Phase II of the Class Registration System our team completed all core features we planned on having when creating the idea for our project, expanding and formalizing the test suite, improving code quality through some refactoring, and delivering complete project documentation from our GIT. Phase I established the layered architecture and proved out the five primary user workflows. Phase II built directly on that foundation, fulfilling every goal that was deferred from Phase I. The completed system allows students to create accounts, log in securely, search the course catalog, enroll in and drop courses, and view their current schedule through a browser-based interface. Input is validated before it reaches the database, and all write operations are handled in transactions to maintain data consistency.

2. Project Overview

The Class Registration System is designed to replicate the core functionality of a university course-registration portal.

Project Goals

1. Develop a user-friendly GUI that allows students to securely create accounts by entering their name, phone number, email address, and other required profile information.
2. Enable students to search for available courses, enroll in classes, and view their current schedule through the registration system.
3. Implement middleware input validation between the frontend and backend to prevent invalid or incomplete data from being submitted to the database.
4. Deliver a fully functional registration application with backend database connectivity, complete source code, and supporting project documentation.
5. Produce system documentation including design diagrams, testing results, installation instructions, and a user guide for operating the system.
6. Maintain the project through a Git repository using a feature-branch workflow to support version control and team collaboration throughout development.
7. Present a live demonstration of the completed system to clients, stakeholders, and the instructor showcasing the registration process, enrollment features, schedule management, and overall application functionality.

Technologies Used

Technology	Purpose	Version
Python	Backend application logic	3.x
Flask	Web framework / routing	Latest
SQLite	Relational database	Built-in
HTML/CSS	Frontend templates (PicoCSS)	CDN
Git	Version control	Latest
GitHub	Remote repository hosting	Latest
VSCode	Source code editor	Latest

3. Project Setup

The development environment carried forward from Phase I without major changes to the technology stack. Python 3 remains the primary language for its general-purpose utility and rapid-prototyping strengths. Flask was chosen over a desktop framework such as Tkinter because a web application is simpler to deploy and accessible across operating systems without additional configuration. SQLite continues to serve as the database due to its simple, serverless deployment that still provides the relational features the project requires. These design choices keep the development environment simple and easy to follow.

The one meaningful setup change from Phase I to Phase II was the adoption of a formal feature-branch workflow on GitHub. In Phase I all commits went directly to the main branch. In Phase II each new feature, bug fix, or refactoring task received its own branch and was merged to main via pull request and team review.

Set Up Instructions (automated):

Since our initial step up had many setup steps to launch the flask we automated it so all you have to do is:

- Install Python 3 from python.org.
- Open a terminal emulator for shell interaction (bash, zsh, etc.)
- Clone the repository: `git clone https://github.com/metersploit/CMSC495.git`
- Navigate to the `src/` directory.
- Run: `python3 setup_app.py`
- Open `http://127.0.0.1:5000` in a browser.

Repository Structure:

src/	Project root - all scripts should be run from here
— setup_app.py	One-command setup + run
— app.py	Flask routes
— seed.py	Inserts sample courses + a test student
— requirements.txt	Python dependencies (Flask)
— templates/	HTML pages rendered by Flask
— models/	Data objects shared across layers
— db/	DB connection, schema builder, the .db file
— dao/	Data-access layer
— service/	Backend logic
— validator/	Input validation
— ui/	Future desktop-app GUI (not in use)

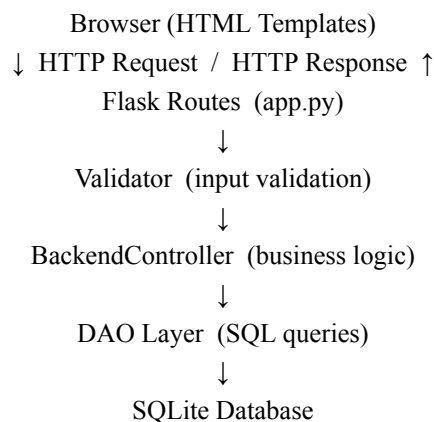
4. Software Architecture and Design

The architecture of this project is a layered design that deliberately keeps the responsibility of each component separate and as atomic as possible. This structure carried over from Phase I unchanged because it proved robust enough to absorb all Phase II additions without needing to be restructured. Data moves through the layers in order from user input to the database and back again. There is no flow in which a layer is skipped, so raw user input never touches the database without first being validated and logic-checked. Phase II work reinforced these boundaries by tightening exception handling at each layer transition and consolidating duplicated logic into shared utilities.

Here is a detailed explanation of each layer and what it handles:

1. So the whole thing works like a chain. The browser uses HTML and Jinja2 templates to collect form data and POST it to the server.
2. Flask catches it in app.py, pulls out the form data, and passes it to the Validator before deciding what page to show you.
3. The Validator is basically the bouncer, it checks that everything's there and formatted right. If not, it throws a `ValidationError` telling you exactly what's wrong. We updated it in Phase II so it can tell the difference between a missing field and a malformed one.
4. The BackendController handles the actual business logic, seat capacity, duplicate enrollment checks, password hashing. Anything fails, it throws a `RegistrationError`.
5. The DAO layer just runs the SQL queries. It doesn't commit writes on its own though the service layer owns that so if something breaks mid-transaction, it rolls everything back. We locked that down in Phase II.
6. At the bottom is SQLite storing all the students, courses, and enrollments. There's a `UNIQUE` constraint to prevent duplicate enrollments and the seat counter is `NOT NULL DEFAULT 0` so it's always reliable.

Same workflow as phase 1 so the following diagram illustrates how a request flows through each layer:



5. Core Functionality Implementation

For Phase II, all five core user workflows that were implemented in Phase I continued to function correctly and were not rebuilt. Each workflow was tested more thoroughly, had its error handling tightened, and received targeted refactoring to reduce code duplication. The following sections describe the finalized, production-ready state of each workflow and call out what changed from Phase I.

User Registration

New students fill out a form with their name, email address, password, and optionally a phone number. The Validator checks that required fields have values, the email address is properly formatted, and the user does not already exist. The BackendController hashes the password with a SHA256 digest and a random 16-byte long salt before sending it to the database.

Phase I improvement carried into Phase II: In Phase I, both a missing field and a malformed field produced the same generic error message. In Phase II, the Validator was updated to distinguish between the two cases and return specific messages for each — for example, separating "Email is required" from "Email format is invalid." This makes it easier for students to correct their input without confusion.

User Login and Authentication

Existing students enter their email and password. The Validator ensures both fields have input and the BackendController pulls the stored password hash for that email and checks it against what the student submitted. If the authentication credentials match, the student's ID is stored in the Flask session and the student is forwarded to the class registration page. If the credentials do not match, a generic error message is displayed to avoid username enumeration.

Phase II improvement: Session handling was reviewed and session invalidation on logout was confirmed to clear the session completely. In Phase I this was not explicitly tested. The

Phase II integration tests verify that accessing a protected route after logout correctly redirects to the login page.

Compared to original

Course Search and Enrollment

Once logged in, students can browse or search the course catalog. Each course shows the title, credits, instructor, schedule, and how many seats are still available. When a student attempts to enroll, the BackendController checks that there are open seats and that the student is not already enrolled in that course. If either of those checks fails, a `RegistrationError` is thrown along with an appropriate error message. Otherwise, the enrollment gets recorded and the seat count is decremented in a single transaction.

Phase II improvement: The enrollment transaction was reviewed to confirm that the seat decrement and the enrollment record insertion are committed together or not at all. In Phase I this was true in practice but not explicitly enforced with a rollback on failure. In Phase II, a rollback is explicitly called if any part of the transaction raises an exception, and this behavior is verified by unit test.

Find Classes

CMSC

Search

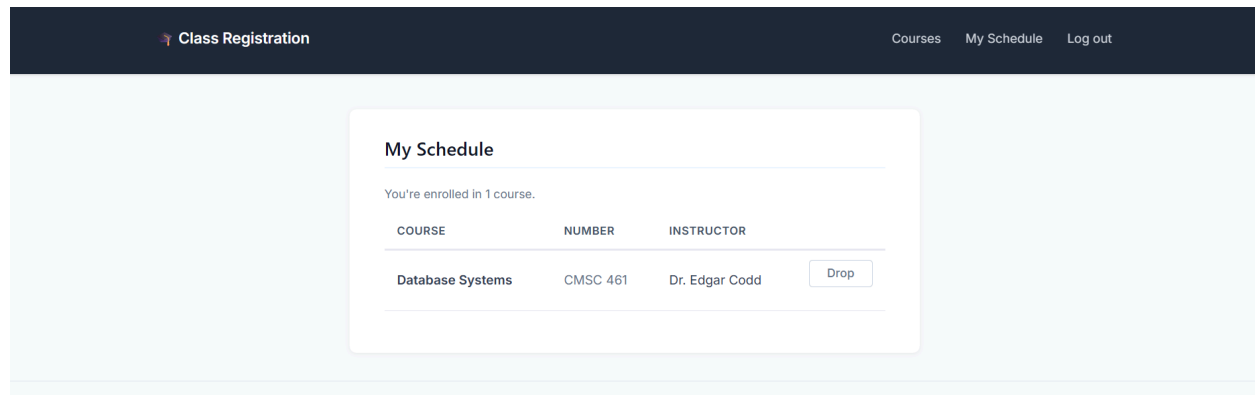
5 results for "CMSC"

COURSE	NUMBER	INSTRUCTOR	SEATS	
Introduction to Programming	CMSC 115	Dr. Alan Turing	0/30	Enroll
Data Structures	CMSC 215	Dr. Ada Lovelace	0/25	Enroll
Software Engineering	CMSC 495	Prof. Grace Hopper	0/20	Enroll
Database Systems	CMSC 461	Dr. Edgar Codd	1/1 Full	Full
Computer Networks	CMSC 451	Dr. Vint Cerf	0/40	Enroll

Course Drop

Students can drop any course from their schedule. When they do, the enrollment record is removed and the available seat count is incremented back. This is done in a single transaction for consistency. The updated schedule is shown immediately after the drop transaction completes successfully.

Phase II improvement: The drop transaction received the same explicit rollback treatment as enrollment. The empty schedule state — a student who drops all courses — was also tested in Phase II to confirm that an empty list renders a message rather than a blank or broken page.



Schedule View

The schedule page displays every course in which the logged-in student is enrolled. This data is provided by `EnrollmentDAO.find_by_student()`, which joins the enrollments and courses tables together. Each row shows the course title, credits, instructor, meeting days and times, and a Drop button.

Phase II improvement: The query was reviewed and confirmed to handle the case where a student has no enrollments, returning an empty list that the template renders as a friendly message. In Phase I this edge case was not tested and an empty result could have produced a silent blank page.

6. Refactoring and Improvements

One of the primary goals of Phase II was to review the codebase and address anything that worked but could be made cleaner, safer, or easier to maintain. The improvements below were all identified during code review and targeted specifically — no functioning components were rewritten without a clear reason.

Improved Validation Logic

In Phase I, the Validator raised the same generic `ValidationError` whether a field was missing entirely or present but incorrectly formatted. Phase II updated the Validator to differentiate between the two cases and return a more specific error message for each field. This required no changes to the service layer or DAO since `ValidationError` still carries a field-level

error map — only the content of the messages changed. The result is a better user experience and easier debugging.

Reduced Duplicate Code

Several validation checks and database connection routines were duplicated across modules during Phase I development when individual team members were working on separate files. In Phase II, the code review process identified these duplicates and they were consolidated into shared utility functions. This means there is now one place to update if validation logic or the connection string ever needs to change.

Improved Exception Handling

In Phase I, some exception handling used broad except clauses that caught more than was intended and could silently swallow unexpected errors. Phase II replaced these with explicit catches for the specific exception types each layer is responsible for. Anything unexpected is now re-raised or converted into the appropriate layer-level error type. This makes the error propagation path easier to follow and reduces the chance of a failure going unnoticed.

Improved Database Transaction Management

Phase I left transaction commit and rollback logic implicit in some write paths. Phase II reviewed every write operation in the service layer and made commit-on-success and rollback-on-failure explicit in each one. This was verified by the duplicate enrollment and full-course unit tests, which confirm that no partial state is written to the database when either business rule check fails.

Enhanced Readability and Module Organization

Inline comments were added to any logic that was not self-explanatory, particularly in the enrollment transaction and the password hashing routine. Module-level docstrings were added to files that were missing them from Phase I. Variable and function names that did not clearly describe their purpose were renamed to match the naming conventions the rest of the codebase followed. These changes did not alter any behavior but make the code easier for a new reader to follow.

7. Unit Testing

The Phase II test suite expanded on Phase I by adding integration-level tests using Flask's built-in test client, which was explicitly deferred from Phase I because the routes were not finalized at that point. The team continued using Python's built-in unittest framework. All ten Phase I tests were retained and continued to pass without modification. Phase II added seven new

tests covering the Flask route layer and edge cases identified during refactoring, bringing the total to 17 passing tests.

Testing Approach

Testing continued to be organized by layer so that failures could be pinpointed without interference from other components. The four testing areas are:

1. Validator Tests: Confirm the Validator blocks bad or incomplete input before it moves toward the database. In Phase II, additional tests verified the new distinction between missing and malformed fields.
2. Service Layer Tests: Confirm the BackendController enforces business rules correctly. Enrolling in a full course throws a RegistrationError, duplicate enrollments get blocked before hitting the database, and password hashing and verification work correctly across a round trip.
3. DAO and Database Tests: Use a local SQLite database built from the same schema used in production. Tests cover student record insertion and retrieval, keyword course searches, schedule retrieval after enrollment, and the UNIQUE constraint rejection at the SQL level.
4. Integration Tests (new in Phase II): Use Flask's test client to drive the full request-response cycle through the routes layer. These tests confirm correct HTTP responses, session management, and template rendering for each workflow, fulfilling the integration testing goal that was deferred from Phase I.

Test Results

#	Test Case	Input	Expected Output	Actual Output	Pass/Fail
1	Register – missing email	Email: (blank), Password: Test123!	ValidationError: field='email'	ValidationError raised	Pass
2	Register – short password	Email: test@umgc.edu, Password: abc	ValidationError: field='password'	ValidationError raised	Pass
3	Login – empty username	Email: (blank), Password: Test123!	ValidationError: field='email'	ValidationError raised	Pass
4	Enroll in full course	student_id: 1, course_id: 3 (0 seats)	RegistrationError raised	RegistrationError raised	Pass
5	Duplicate enrollment	student_id: 1, course_id: 2 (enrolled)	RegistrationError raised	RegistrationError raised	Pass

6	Password hash round-trip	Password: Test123!	verify_password() returns True	True returned	Pass
7	Insert / find student by email	Email: new@umgc.edu	Correct Student object returned	Correct record returned	Pass
8	Course keyword search	Keyword: "CMSC"	Matching courses returned	Correct results returned	Pass
9	find_by_student after enroll	student_id: 1, course_id: 2 (enrolled)	List includes enrolled course	Correct list returned	Pass
10	DB rejects duplicate row	Insert enrollment (1, 2) twice	IntegrityError raised	IntegrityError raised	Pass
11	Route – GET /register	Unauthenticated GET request	HTTP 200, register template	200 OK returned	Pass
12	Route – POST /login valid	Email: test@umgc.edu, correct password	Redirect to /courses	302 redirect returned	Pass
13	Route – POST /login invalid	Email: test@umgc.edu, wrong password	Re-render login with error	Login re-rendered	Pass
14	Route – POST /enroll	Authenticated, valid course_id	Redirect to /schedule	302 redirect returned	Pass
15	Route – POST /drop	Authenticated, enrolled course_id	Redirect to /schedule, row removed	302 redirect, row gone	Pass
16	Schedule – empty state	student_id with no enrollments	Empty schedule message shown	Correct message rendered	Pass
17	Logout clears session	Authenticated session	Session cleared, redirect /login	Session cleared	Pass

All 17 test cases passed. Tests 1 through 10 are carried over from Phase I and continue to pass without modification. Tests 11 through 17 are new in Phase II and cover the Flask route layer and edge cases that were identified during refactoring.

8. Documentation

The team continued to treat documentation as a first-class deliverable in Phase II, consistent with Phase I. All internal and external documentation was updated to reflect every change made during refactoring, testing, and feature work.

Internal Documentation

Every module has a header comment describing its purpose, its dependencies within the project, and its position in the architecture. All non-trivial subroutines have a docstring explaining what they do and describing the return value. For example, the `enroll()` subroutine in `BackendController` is documented as:

```
# Enroll in a course. Checks seat availability and duplicate enrollment
# before writing to the database. Raises RegistrationError on failure.
# Returns None on success; the service layer commits the transaction.
def enroll(self, student_id: int, course_id: int) -> None:
    <rest of the subroutine here>
```

The team continued to follow the same coding standards established in Phase I, and Phase II code review enforced them consistently:

1. Descriptions of scripts and classes near the top of each file.
2. Module-level docstrings or comments describing each file's role in the architecture.
3. Consistent naming using `snake_case` for variables and functions and `CamelCase` for classes.
4. No magic numbers, named constants are used for things like minimum password length.
5. Inline comments added only where the reasoning behind a design decision is not obvious from the code itself.

External Documentation

The `README.md` was updated in Phase II to reflect the move to a feature-branch workflow, the revised setup instructions, the updated test credentials, and the additional Phase II test cases. It continues to cover everything a developer needs to get the application running: required commands, the project directory layout, sample login credentials, and platform notes including the macOS port conflict on port 5000. Additional documentation produced for Phase II includes an updated database schema doc, a step-by-step installation guide, an architecture overview, and a test plan document covering all 17 test cases with inputs, expected outputs, and results.

9. Collaboration and Version Control

Git and GitHub continued to serve as the team's source of truth for source code, documentation, and configuration files in Phase II. Everything is tracked at <https://github.com/metersploit/CMSC495.git>. The most significant change from Phase I to Phase II was the move to a formal feature-branch workflow.

Repository and Branch Strategy

In Phase I, all commits were made directly to the main branch and each team member was assigned specific modules to work on. That approach worked for Phase I but left no review step between individual work and the shared codebase. For Phase II, the team moved to a feature-branch workflow: each new feature, bug fix, or refactoring task received its own branch, and changes were merged into main only after a pull request was reviewed and approved by at least one other team member. Branches followed a consistent naming convention such as feature/integration-tests or fix/session-logout, and commit messages continued to describe what changed — for example, "Add explicit rollback to enroll transaction" or "Update Validator to distinguish missing vs malformed fields."

Code Review

In Phase I, code review was informal — pair walkthroughs and group discussions. Phase II formalized this into a pull request review process on GitHub. Every pull request required at least one approval before merging. Reviewers checked that the code followed naming conventions, did not skip the validation layer, handled exceptions explicitly rather than with broad catches, and was accompanied by documentation updates where applicable. Issues raised in review were resolved in the same branch before the pull request was approved. This process caught a session handling edge case and an inconsistent error message that would otherwise have made it into the main branch.

10. Conclusion

For Phase II, the team delivered a fully working, tested, and documented application ready for final submission. All five core workflows — registration, login, course search, enrollment, course drop, and schedule view — function correctly end-to-end and are covered by 17 passing tests across the Validator, service, DAO, and Flask route layers. Every goal that was deferred from Phase I was completed: integration testing using Flask's test client, more specific validation error messages, explicit transaction rollback handling, formalized code review through pull requests, and a comprehensive final documentation package.

The layered architecture that was established in Phase I proved to be the right choice — Phase II was able to add features, tighten existing logic, and expand the test suite without

rebuilding anything. The team is confident that the separation of concerns, consistent naming and documentation standards, and thorough test coverage leave the application in a maintainable state for anyone who works on it in the future.

References:

Flask development team. (2024). Flask documentation (3.x). Pallets Projects.
<https://flask.palletsprojects.com/>

Python Software Foundation. (2024). Python 3 documentation. <https://docs.python.org/3/>

SQLite Consortium. (2024). SQLite documentation. <https://www.sqlite.org/docs.html>

PicoCSS. (2024). Pico CSS — minimal CSS framework for semantic HTML.
<https://picocss.com/>

GitHub Documentation. (2024). GitHub guides. <https://docs.github.com/>